Projeto do Robô — Entrega 1

Responsável: [preencher com o nome do estudante]
Disciplina: Algoritmos de apoio à pesquisa operacional
Linguagem: Python 3.10+
Versão do código: 1.0.0

Resumo

Este artigo descreve a implementação da primeira entrega do Projeto do Robô. O problema consiste em encontrar o menor custo para atravessar uma grade desde o canto superior esquerdo até o canto inferior direito, podendo mover-se somente para a direita ou para baixo e sem entrar em obstáculos. A solução principal usa recursão sem memoização: cada posição calcula os custos dos dois movimentos possíveis e escolhe o menor caminho viável. Também foi implementada uma estratégia gulosa para comparação experimental. Os testes confirmam o contrato de entrada e saída, os resultados fornecidos no enunciado e a diferença de qualidade entre a busca recursiva e a regra gulosa.

Palavras-chave: recursão; caminho mínimo; grade; algoritmo guloso; testes.

1. Descrição do problema

A entrada oficial é um texto JSON contendo a chave `custos`. O valor dessa chave deve ser uma matriz não vazia, formada por linhas não vazias de mesmo tamanho. Cada posição contém um inteiro positivo, que representa o gasto de energia da casa, ou `null`, que representa um obstáculo.

O robô começa na posição `[0, 0]` e precisa chegar à posição da última linha e da última coluna. A cada passo, ele pode avançar uma casa para a direita ou uma casa para baixo. Ele não pode sair dos limites da matriz nem atravessar obstáculos. O custo de uma rota é a soma dos valores de todas as casas visitadas, incluindo a partida e a chegada.

A função pública `resolver(entrada)` devolve um objeto Python com o contrato equivalente ao JSON do enunciado:

- quando há rota, retorna `{"status": "ok", "custo": valor}`;
- quando a entrada é válida, mas não existe rota, retorna `{"status": "sem_caminho", "custo": None}`;
- quando a entrada é inválida, retorna `{"status": "entrada_invalida", "erro": mensagem}`.

A lista de coordenadas do caminho e a memoização não foram incluídas, pois pertencem à Entrega 2.

1.1 Exemplo válido sem rota

```
{"custos": [[1, null], [null, 1]]}
```

A partir da casa inicial, o robô não pode ir para a direita nem para baixo. Portanto, a saída é:

```
{"status": "sem_caminho", "custo": null}
```

1.2 Exemplo inválido

```
{"custos": [[1, 2], [3]]}
```

As linhas possuem tamanhos diferentes, portanto a matriz não representa uma grade válida. A saída possui status `entrada_invalida` e uma mensagem explicativa.

2. Implementação recursiva

A função `recursivo(mapa, linha, coluna)` resolve o problema a partir de uma posição. Há quatro situações fundamentais:

1. uma posição fora da grade é impossível;
2. uma posição com `None` é um obstáculo e também é impossível;
3. na chegada, o menor custo é o custo da própria casa;
4. nos demais casos, a função calcula os caminhos pela direita e por baixo, descarta os impossíveis e soma o valor da casa atual ao menor custo restante.

O valor `None` é usado internamente para representar um ramo sem solução. Assim, quando os dois vizinhos são impossíveis, a posição atual também não oferece uma rota.

A implementação não armazena resultados de chamadas anteriores. Se dois ramos chegam à mesma posição, a posição é calculada novamente. Esse comportamento é intencional e atende à exigência da Entrega 1; a memoização será uma possibilidade futura.

A função `resolver` faz a leitura do JSON com a biblioteca padrão `json`, valida a estrutura e chama `recursivo`. A validação rejeita listas vazias, linhas vazias, linhas de comprimentos diferentes, booleanos, textos, números reais, zero e números negativos.

3. Testes

Os testes foram escritos com `unittest` e podem ser executados isoladamente com:

```
python -m unittest discover -s tests -v
```

Foram cobertos:

- matriz `1 x 1`;
- matriz de uma linha e de uma coluna;
- matriz `2 x 3`;
- o mapa original do enunciado, com resultado 11;
- uma matriz com obstáculos, mas ainda alcançável, com resultado 6;
- mapas sem caminho;
- listas vazias, linhas vazias e linhas de tamanhos diferentes;
- valores inválidos, incluindo `true`, texto, número real, zero e número negativo;
- JSON sem a chave `custos` e JSON malformado;
- três casos adicionais criados para este trabalho;
- contagem de chamadas recursivas;
- comparação entre a solução recursiva e a gulosa.

A execução da versão 1.0.0 produziu **7 testes aprovados**.

4. Por que a solução funciona

4.1 Término

A cada chamada válida, o robô avança uma linha ou uma coluna. Como os movimentos permitidos somente aumentam a linha ou a coluna, não é possível voltar à mesma posição. Em uma grade finita, depois de no máximo um número finito de movimentos o robô chega à casa final ou todos os ramos tornam-se impossíveis. Por isso a recursão sempre termina.

4.2 Correção do menor custo

Toda rota válida que parte de uma posição não final precisa começar pela direita ou por baixo. A função calcula recursivamente o menor custo possível em cada uma dessas duas alternativas. Como ela considera os dois ramos, não perde uma rota melhor. Depois, escolhe o menor ramo viável e soma o custo da casa atual. No caso-base, a chegada contribui apenas com o próprio custo, exatamente como define o enunciado. Por indução sobre a quantidade de movimentos restantes, o resultado é o menor custo da posição atual até a chegada.

4.3 Ausência de rota

Se uma posição não é a chegada e os dois movimentos seguintes são impossíveis, a função retorna `None`. Esse valor se propaga para os chamadores. Se o início também não encontra nenhum custo possível, `resolver` transforma o resultado em `{"status": "sem_caminho", "custo": null}`.

5. Medição do trabalho

A função `contar_chamadas` usa um contador externo apenas para observar quantas chamadas válidas foram feitas; ele não guarda respostas e não altera a busca. As chamadas para posições fora da grade não são realizadas, pois a função verifica os limites antes de chamar os vizinhos.

Grade	Chamadas obtidas	Chamadas esperadas
2 × 3	9	9
3 × 3	19	19
6 × 6	923	923
10 × 10	184.755	184.755
1 × 10	10	10
Mapa do enunciado	43	—

Nas grades preenchidas com custo 1, os valores coincidem com o enunciado. O crescimento é elevado porque a mesma subgrade é recalculada em diferentes ramos. Em uma grade de uma única linha, só existe um caminho e, por isso, ocorre uma chamada por casa. Já em grades com várias linhas e colunas, a árvore de chamadas cresce rapidamente. Essa é precisamente a motivação para estudar memoização na próxima entrega.

6. Regra gulosa

A função `guloso(mapa)` escolhe sempre o vizinho livre com menor custo imediato. Quando há empate, a ordem dos candidatos faz com que a direita seja escolhida antes de baixo. A função soma os custos visitados e retorna `None` quando fica sem movimento possível.

No mapa do enunciado, a estratégia gulosa produz custo **26**, enquanto a busca recursiva encontra custo **11**. A decisão local não considera que uma casa aparentemente barata pode conduzir a uma sequência de casas caras.

6.1 Contraexemplo próprio

Foi usada a grade `3 x 3`:

```
1   1  50
5  50  50
1   1   1
```

A solução ótima desce pela primeira coluna e depois segue para a direita:

```
1 → 5 → 1 → 1 → 1
```

O custo é `1 + 5 + 1 + 1 + 1 = 9`.

A regra gulosa escolhe a direita na primeira decisão, porque o custo 1 é menor que 5. A partir daí, ela segue por casas caras:

```
1 → 1 → 50 → 50 → 1
```

O custo guloso é `103`. A escolha local de custo 1 na primeira etapa impediu o acesso à coluna de baixo custo e mostrou que a regra gulosa não garante o ótimo.

7. Organização e execução

O código está organizado da seguinte maneira:

- `src/robo/solucao.py`: validação, busca recursiva, estratégia gulosa e medição;
- `src/robo/cli.py`: leitura de arquivo ou entrada padrão e impressão do resultado em JSON;
- `tests/test_solucao.py`: testes automatizados;
- `exemplos/mapa_exemplo.json`: exemplo de entrada do enunciado;
- `README.md`: instruções de instalação e uso;
- `CONTRIBUTING.md`: convenções de commits e comentários.

Para executar o exemplo sem instalar dependências externas:

```
PYTHONPATH=src python -m robo.cli exemplos/mapa_exemplo.json
```

A saída é:

```
{"status": "ok", "custo": 11}
```

8. Conclusão

A implementação atende ao contrato da Entrega 1 em Python, utiliza recursão sem guardar resultados anteriores, identifica entradas inválidas e situações sem caminho, e apresenta testes reproduzíveis. A medição evidencia o custo da abordagem sem memoização. A comparação com a regra gulosa também mostra por que escolher o vizinho mais barato isoladamente não é suficiente para resolver o problema de forma ótima.